altairsim — Changelog

2026-09-05 · 0a9afc5

A simulation of the MITS Altair 8800 and the S-100 bus

Changelog

Every release of **altairsim**, newest first. Each entry is the short version — what you can do here that you could not do in the release before it. The User Manual describes the program as it is now; this document is the record of how it got there.

Unreleased

Documentation

A **migration guide** for people coming from the two best-known open Altair/S-100 simulators — AltairZ80 (SIMH) and z80pack: what carries across untouched, an objective side-by-side, the command mappings (including the deliberate SIMH `D` / `E` swap), disk-image compatibility, and worked machine-file conversions (`docs/migrating.md`).

1.0.0

1.0.0 is the version that says the simulator is what it set out to be. Where 0.4.0 filled the backplane, 1.0 fills out the processors behind it and the world around it: two more CPU cores — including the first that is not an Intel — more S-100 boards and the machines they boot, a disk that arrives over the network, and an AI assistant that can now drive a machine in real time against real hardware. It is the release the earlier ones were building toward.

Two more CPU cores — and the first that isn't an Intel

The **Motorola 6800** joins the 8080 and Z80, and with it the **MITS Altair 680b** — a different computer from the 8800, a different bus, a different instruction set. `altairsim altair680` brings it up under **MON680**, the 680b's own monitor ROM, and `DISASM` and `EDIT` speak 6800 the way they speak 8080 where that core is active, down to a 6800 assembler for entering code a byte at a time. Its serial I/O board (`680io`) carries the console, the Universal I/O board (`680uio`) adds a second port, and the **KCACR cassette** board loads and saves off tape — enough of the machine that MITS **680 BASIC** loads from cassette and runs. Because Motorola tools speak Motorola formats, `LOAD` now reads a **Motorola S-record** (`.S19`) file as well as Intel HEX, and takes an explicit `FORMAT=SREC`.

The **Intel 8085** joins them too — the 8080's binary superset, with `RIM` / `SIM` and the on-chip interrupt system (the non-maskable `TRAP` and the maskable `RST 5.5/6.5/7.5`, each with its mask and pending latch, in hardware priority order) that the 8080 never had. It is faithful where the two chips actually

differ, down to the two undocumented condition bits — **V** (signed overflow) and **K** — and all ten undocumented opcodes, which execute rather than trap. Like the other cores it earns its place at the gate rather than by inspection: Ian Bartholomew's 8085 exerciser, whose expected CRCs were read off real silicon, runs its 2.9 billion instructions against it on every CI push, on all three platforms. `altairsim 8085` is the direct analog of the `z80` machine — an 8085, 64K, and a 2SIO console — and the disassembler and assembler speak 8085, undocumented opcodes flagged the way `DDT` flags a byte outside the published set (`??= 08  *DSUB`).

More boards, and the machines they boot

Five more controllers and their operating systems boot alongside the rest:

- The **iCOM FD3712/FD3812** 8″ floppy controller — a programmed-I/O command engine with its driver in a high-memory boot PROM — boots CP/M 2.2 in single and double density and both revisions of iCOM's own **FDOS** disk operating system (`altairsim icom`).
- **S100Computers**' modern reproduction boards boot **CP/M 3**: the **Dual SD** controller runs two microSD cards as raw drives, and the **IDE-AB** board runs a **CompactFlash** card as drives A:/B:; because both lay a card out identically, one system image is interchangeable between them, and the combination board spans all four drives (`altairsim dualsd`, `altairsim dualide`, `altairsim dualidesd`).
- The **CompuPro System Support 1** is a real-time clock, a serial channel, an interval timer and a pair of cascaded interrupt controllers on one card — its OKI MSM5832 reads your host's own date and time and survives a RESET, and everything but the empty math-chip socket is implemented (`altairsim compupro`).
- The **SSM PB1** is a **PROM burner**: you *run* a period EPROM-programmer routine against a board that presents the real socket-and-arm interface, burn a 2708 or 2716, and `SAVE` the result as Intel HEX. `examples/pb1` carries SSM's own driver routines from the PB1 manual.

The **Tarbell #2022** now boots a disk whose CBIOS moves sectors by **DMA**, driving the card's on-board Intel 8257 to steal S-100 bus cycles and drop each sector into memory itself — the first board in the simulator to master the bus (`altairsim tarbelldd-dma.toml`). **Bank-switched RAM** is now its own board, `bankmem`, modeling four real decoders (Vector, Cromemco, North Star, and the SD Systems **ExpandoRAM II**), and on the ExpandoRAM II an SD Systems machine boots **banked CP/M 3** with a full 48K TPA under the operating system's own bank. And the **SSM 8080 System Monitor V1.0** is now a built-in ROM the way the Eberhard monitors are — boots by name, nothing to fetch — with a cheatsheet to match.

Two new serial boards land the SSM cards properly. **Generic SIO** (`gsio`) is a describe-it-by-strap serial card: you say where the status/control and data ports sit, which status bits carry data-available and transmit-empty, and whether the shared inverter gate is engaged, and that is the port — with two independent channels and built-in profiles that imitate the MITS SIO Rev 0, the Cromemco TU-ART, the IMSAI SIO-2 and the CompuPro channels. And the fully emulated **SSM IO-4** (`io4`) models the actual

2P+2S card — its two 1602-family UARTs with real programmable word length, parity and stop bits; a strappable status word matching the W1/W2 header; four 8212 latched parallel ports; and header-W4 interrupts, where each serial receive and transmit and each parallel input can be strapped onto any vectored-interrupt line. It boots the SSM 8080 monitor as its console.

A disk that arrives over the network

`MOUNT` now takes a **`tnfs://host/path`** URL and fetches the image off a **TNFS server** — the network file system the FujiNet project speaks — instead of reading a file on your host. Once mounted it behaves like any other disk: the guest reads and writes it, `WP` protects it, and your changes are written back to the server. It works for every disk and tape controller, since it is the image that arrives over the network, not anything the board can tell apart from a local file. And because a network can vanish mid-session, altairsim now says so out loud if the server stops accepting writes and keeps retrying, rather than letting your changes pile up unsaved in silence.

Driving a machine over MCP got real teeth

An AI assistant driving a machine over `--mcp` no longer has to screen-scrape the text monitor to inspect it. The interface gained **twelve first-class, typed tools** for the work it used to reach through the monitor prompt by hand — `step` and `breakpoints`, `snapshot` / `restore`, the always-on `bus_trace` flight recorder, `mem_fill` / `mem_search` / `mem_save`, a stateless `disasm` that decodes a ROM with no CPU running, typed `mount` / `connect` wiring, and a `bus_irq` view of who is pulling the interrupt lines — bringing the MCP surface to the parity the design lays out, structured data in and out where there used to be a prompt to parse.

And two things it could not do at all, it can now. It can **share a session**: `--mirror` opens a live bidirectional socket onto the same console the assistant is driving, so a person can watch what it is doing and take over the keyboard, then hand it back. And it can work with **real hardware in real time** — wire a serial line to an actual device and set a real clock speed, and the `run` tool paces the guest to that clock instead of running flat out, so a reply that takes the device a fraction of a second arrives while the guest is still waiting for it; `run` no longer cuts a transfer off when its time budget runs out, so a boot loader pulling its whole system image in over a serial disk finishes in a single call. The console's own text transforms now ride along under `--mcp` and `--mirror`, so what the assistant reads — and what a watcher sees — matches what a person at the real terminal would, parity bit and carriage returns and all.

A terminal in its own window

A serial line can now open its own **built-in terminal window** the simulator draws itself — `CONNECT sio0:a terminal`, or `connect = "terminal"` in a machine file — so the machine's console and the monitor's command prompt live in separate windows with no telnet client and nothing to install. It speaks four dialects the way period software expects them, `?emulation=` picking one: **VT100/ANSI** (the default), the CP/M **ADM-3A**, the **VT52**, and the Heath/Zenith **H19** — the last three being terminals no modern

emulator provides, which was the whole point. It draws in the real **DEC VT220** character set decoded from the terminal's own character ROM, on a softened green phosphor (or `?phosphor=amber`), and carries the same fold-the-bytes settings the console has in a `[terminal]` section — which earn their keep on a period even-parity monitor that computes parity into bit 7. In a headless build the endpoint refuses cleanly rather than opening a line nobody can see.

The video window

`SET DISPLAY crt=on` (or `[display] crt = true`) paints any video window like the period tube — the 4:3 aspect of a real monitor, the raster softened into a phosphor glow rather than a grid of hard pixels — and under that look a window opened at a chosen `width` fills exactly that many pixels. Each video board now opens its **own** host window, so a machine with two video cards shows two pictures at once rather than sharing one.

The debugger, and small conveniences everywhere

Cycle breakpoints now take a condition: `BREAK MEM W 100 IF B==0` stops only on the access whose registers hold, and `BREAK IO R 10 LOADS A>7F` tests the byte an `IN` actually read rather than the state it read it with. `EXAMINE <addr>` now shows the register line and the next instruction along with the byte. The byte-at-a-time `EDIT` assembler learned the **Z80**, so between the new cores above and this, `EDIT` and `DISASM` now speak whichever of the four processors is running. And a scatter of the prompt's rough edges are smoother: a leading `~` in a typed path expands to your home directory, monitor subcommands abbreviate by unique prefix, `LOAD` reports the page count of a CP/M `SAVE`, the host serial layer accepts non-standard baud rates like 76800, and the Sol-20's `MODE SELECT`, `CLEAR` and `LOAD` keys are reachable from the keyboard.

The package, and the documents in it

The monitor prompt and the debugger are now their **own shipped PDFs** — `altairsim-monitor.pdf` and `altairsim-debugger.pdf` — pulled out of the manual so each reads as the reference it is. Every PDF in the package is now paginated with a cover, a page-numbered table of contents and running page numbers. And, quietly, MSVC's `/W4` warnings now fail the Windows build the way the other two toolchains already did, closing the last corner where a warning could slip through unseen.

0.4.0

0.4.0 is the boards release. Seventeen new S-100 cards join the backplane — enough that three whole new machine families (Cromemco, SD Systems, Tarbell) boot alongside the MITS originals for the first time — disks and cassettes learn to be built and formatted by the guest instead of only read, and the debugger, the monitor prompt and the video window all got noticeably sharper along the way.

Three new machine families join the MITS originals

Cromemco arrives as a full boot chain: the **16FDC/64FDC** floppy controller (Western Digital FD1793, a TMS5501 console UART, and the RDOS boot PROM in one card) boots **CDOS**, and the **Dazzler** paints S-100's first color graphics out of a framebuffer in main RAM, with the **D+7A** analog/parallel card and a **JS-1 joystick** (real gamepad or keyboard) to drive games on it. **SD Systems** shows up as a matching trio — the **SBC-100/200** single-board Z80 (an 8251 console that auto-bauds to your terminal, and later a full interrupt-driven CP/M boot with the onboard PROM switching itself out), the **VersaFloppy** WD177x controller booting SDOS, and the **VDB-8024** 80×24 video terminal card. The **Tarbell #1011** and **#2022** floppy controllers boot CP/M entirely on their own, no monitor involved, straight off their own boot PROM. On the MITS side, the **88-HDSK** boots CP/M off a hard disk, **88-PIO/88-4PIO** add parallel I/O, **88-LPC** drives a real line printer, **88-UIO** combines a serial port and a cassette deck on one card, and the **8800b Turnkey Module** brings up a front-panel-less Altair with its boot PROM jammed onto the bus at RESET. Two boards round it out: the **PMMI MM-103**, the first S-100 modem (dial and answer a real phone line over TCP), and **usio**, a serial card you describe by strap instead of one we picked, with built-in profiles for the Cromemco TU-ART, IMSAI SIO-2 and CompuPro serial channels. A new built-in ROM, **ROM BASIC**, boots Altair BASIC 4.1 straight out of PROM with the full 48K free underneath it.

Disks and tapes the guest can build, not just read

A hard-sector disk or a cassette tape can now be created **blank** and brought to life by the guest's own software: `MOUNT … CREATE` writes an empty image, and it grows one sector — or one byte of tape — at a time as the guest's FORMAT program defines it, exactly as unformatted media behaves on real hardware. Soft-sector disks caught up too: the Tarbell and VersaFloppy controllers now honor the WD177x Write Track command, so CP/M's `FORMAT` and SDOS's disk formatter lay down a bootable disk from nothing, across every geometry those cards support. `MOUNT` also now reads **ImageDisk (`.IMD`)** files directly, converting the interleaved container to the raw sector-linear image the disk boards expect. On the cassette side, both cassette decks show where the head is (`mm:ss / total (percent)`), a new `WIND` command moves it to a time so a multi-program tape is finally navigable, a `stop` mark parks playback at a boundary, and `EXTRACT` splits a multi-program recording into one file per program. Tapes the simulator writes now load on a real Sol-20, because the cassette modem's output is modeled the way the real hardware's clock-divider and filter actually shape a signal, not as a clean oscillator a real deck could never produce.

A debugger with sharper eyes

Cycle breakpoints (`BREAK MEM` / `BREAK IO`) now stop **before** the triggering instruction runs instead of after, so a breakpoint on a port read shows you the registers exactly as they were the instant the instruction fired — no port read, no byte written, nothing to unwind. `BREAK TAPE STOP` adds a third kind of breakpoint, alongside PC and bus-cycle stops, that halts the instant a cassette reaches its own auto-stop.

`HISTORY` now defaults to a flight recorder of CPU instructions rather than raw bus cycles, and its bus view (`HISTORY BUS`) names which board drove and which answered every cycle it logs. `EDIT`, the byte-at-a-time memory editor, now assembles a full instruction where a byte would go. Loaded symbols make disassembly and single-stepping read by name instead of address, `SAVE` can write a disassembly or octal listing to a file, and the monitor can display and parse addresses, ports and bytes in authentic split **octal**, MITS's own notation. A new diagnostic-channel facility (`SET <id> DEBUG=`, `SHOW DEBUG`) lets individual boards and shared chips narrate what they're doing — a disk stepping heads, a UART taking a byte — each line stamped with the PC that caused it, aimed at the console, a file, or a `log` that tees your whole session transcript to disk. `SET BUS UNCLAIMED` catches a guest reaching for a board that was never fitted, and a bare `!` drops you to your host shell without losing the machine's state underneath it.

A prompt that helps you type

`Tab` now completes commands, board ids, unit names, property names and their legal values at the monitor prompt, reading the candidates straight off the running machine so a board you plug in is completable immediately. The line editor grew word motion, `Home`/`End`, and kill-to-end-of-line, and command history now survives a restart, saved per project directory. Asking what you can build is one family of commands now — `SHOW BOARDS`, `SHOW BOARD <type>`, `SHOW MACHINES`, `SHOW MACHINE <name>` — with legal values listed under every property. A machine file can leave you a note: a `#>` comment line prints itself to the operator when the machine loads, the one or two sentences a config's author needs you to see before you start typing. And `HELP` now answers at whatever level you actually asked for, instead of always dropping you at the top.

The video window and the joysticks behind it

A video window now opens locked to its picture's aspect ratio and resizes proportionally from the first drag, with an even bezel on all four sides and its title bar reading "simulator stopped" whenever the guest is halted. How big it opens is a per-board `width` in pixels, so each card sizes itself off its own resolution; the Dazzler's tiny frame gets its own auto-scaling so it lands near the same size as a VDM-1's instead of a sixth of it. The D+7A now reports what's actually behind each joystick console — a named gamepad, the keyboard, or nothing — and a new `SHOW JOYSTICKS` lists every controller the host can see; two consoles default to two different gamepads automatically, so a pair of controllers just works with no configuration at all.

Save a machine, and pick it back up later

`SNAPSHOT <file>` writes the whole machine's state — every board's registers and RAM, the clock, and the CPU down to the microstate a register dump never shows — to a small, checksummed file, and `RESTORE <file>` loads it back into a machine of the same shape, refusing a corrupt or mismatched file before it touches a single byte of the machine you're running.

Building, driving and reading altairsim got easier

A clean clone now goes from nothing to a running binary with one command, `build.sh` or `build.bat`, with SDL3 staying entirely optional. Every warning the compiler can find now fails continuous integration outright, so nothing that used to be quiet creeps back in unnoticed, and a local sanitizer build is one flag away when a bug needs one. The AI-driving guide gained the step it was missing — how to actually register altairsim as an MCP server — plus a worked example where an assistant assembles a small buggy program, single-steps it to find the fault, and fixes it, entirely over the wire. And a font fix means copying a line out of any built PDF — the manual, an example's README, this changelog — now pastes back as the words it actually says, not text with spaces jammed into the middle of them.

0.3.0

0.3.0 adds no machines and no boards. It changes one thing, and it is the thing the manual has described all along: the copy you download now opens the video window.

The window the manual documents is finally in the package

Every release through 0.2.0 shipped a **headless** binary. SDL3 was not compiled into it, so the VDM-1 and Sol-20 windows the boards and configuring chapters describe at length could not open from anything you were handed — the machines ran, and drew nothing. `SHOW VERSION` said so in a row nobody had reason to read:

```
altairsim> SHOW VERSION
  altairsim  0.3.0
  video      SDL3 -- windowed        <- 0.2.0 and before, the download read: none -- headless
  commit     v0.3.0
  tree       clean
```

0.3.0 is the first release whose downloaded package reads `SDL3 -- windowed`. Run `altairsim sol20`, and a window opens. That is the headline, and most of the rest of this entry is how it was made true on every platform at once.

Nothing to install, on any of the four

The packages are built on the hardware they target now, rather than assembled by CI, and SDL3 is **linked statically into the binary**. So there is no `SDL3.dll` to sit beside the `.exe`, no `.framework`, no `libSDL3.so.0`, and nothing to install before the program runs — the claim altairsim has always made for itself is now true of its video too. On Windows the C runtime is static as well, so a clean machine that has never had a compiler on it runs the `.exe` with no Microsoft redistributable to chase.

The one visible cost is a single extra file in the archive — `LICENSE-SDL3`, SDL3's zlib licence, because SDL3's code now travels inside the binary and its licence travels with it.

macOS ships as two builds, each tested on its own hardware

0.2.0 shipped one *universal* macOS archive. 0.3.0 ships two — `altairsim-0.3.0-macos-arm64` and `altairsim-0.3.0-macos-x86_64` — and the reason is honesty, not size. The universal binary's Intel half had been built and tested by nobody, because the machine that produced it was Apple Silicon; the previous two releases said so in their own notes. Each of the two archives is now built **and** run on the architecture it targets, so the Intel download is exercised on an Intel Mac before it ships. Take the one that matches your Mac; `altairsim --version` and the download filename both name the architecture.

Windows, proven end to end

The Windows package is built with MSVC, links SDL3 and the C runtime statically, passes the full test suite on the machine that builds it, and opens a real VDM-1 window that a person has sat in front of. `dumpbin /dependents` on the shipped `.exe` shows system DLLs only — no `SDL3.dll`, no `VCRUNTIME140`. It is held to the same bar as the other three, and it is no longer an asterisk on the release.

The four downloads

Platform	File
macOS Apple Silicon	`altairsim-0.3.0-macos-arm64.tar.gz`
macOS Intel	`altairsim-0.3.0-macos-x86_64.tar.gz`
Linux x86_64	`altairsim-0.3.0-linux-x86_64.tar.gz`
Windows x86_64	`altairsim-0.3.0-windows-x86_64.zip`

Each holds the program, this changelog, the **User Manual**, `DRIVING-WITH-AI.md`, both licences, and `examples/` — four machines that boot, media included. Unzip it and run it; nothing needs fetching first.

What did not change

The simulator itself is byte-for-byte the machine 0.2.0 was — the same machines, the same boards, the same two CPU cores. The holes named in the manual's introduction are still holes: no snapshot, no replay, the six reserved monitor verbs still reserved, still no audio. Everything that moved is in the box the program arrives in.

0.2.0

The second release. It added machines and made the video window behave like a window — but note that the packages were still **headless** (see 0.3.0): everything below was true of a build made *with* SDL3, which is not what the archives carried until 0.3.0.

Three more monitors that boot from a bare command line

`amon`, `acuter` and `cdbl` became machines, so Martin Eberhard's Altair ROMs boot by name — nothing to fetch, nothing to mount:

```
altairsim amon          AMON 3.1 in a 4K EPROM at F000 -- a full-featured Altair monitor
altairsim acuter        ACUTER at F000 -- CUTER on a plain Altair, driving a terminal
altairsim cdbl          the default machine, with the Combo Disk Boot Loader in the socket
```

The ROM images shipped in 0.1.0 already; what was new is that each got a machine built around it — the whole distance between shipping an image and being able to run it. `hdbl` was deliberately left out: it boots an 88-HDSK hard disk, and there is no 88-HDSK board here.

The video window behaves like a window

- **It does not steal the keyboard when it opens.** The terminal keeps the keys while you type at the monitor prompt; `SET DISPLAY focus=on` hands them to the guest, stopping it hands them back.
- **It is named after the machine**, so `sol20` and `vdm1` are two windows you can tell apart.
- **It is sized to fit the screen it opened on**, and **arrows and HOME reach the guest**.
- **Closing it stops the guest**, instead of leaving a machine running with nothing to draw on.

Two of those were bugs worth naming: typed input could lag a whole frame, and the VDM-1 could repaint hundreds of times per emulated millisecond. Both gone. The VDM-1's cursor now blinks on the board's own oscillator — wall-clock time, as the hardware did.

Disk BASIC, and smaller things

- `examples/diskbasic` boots **Altair Disk BASIC 4.1** off a floppy, media included — a fourth worked example alongside CP/M, cassette BASIC and the Sol-20.
- A binary now names the commit it was built from: `SHOW VERSION` and `--version` carry it, so a report against a nightly or a CI artifact can be traced to the code that produced it.
- `writeprotect` is accepted wherever `readonly` is, in machine files and at `SET`.
- The manual and the program say **board**, not card.

0.1.0

The first release — a simulator for the MITS Altair 8800 and the S-100 bus, in C++20, with **nothing to fetch**: the TOML parser, the JSON encoder and the line editor are all in-tree, so a fresh clone builds with a C++20 compiler and CMake and no network.

Two validated CPU cores

Both cleared their gate before a single board was built on them, and for the release the exercisers were re-run on all three platforms — roughly 15 billion instructions each:

- **8080** — TST8080, 8080PRE, CPUTEST, 8080EXM
- **Z80** — ZEXDOC, ZEXALL

Boards and machines

88-2SIO · 88-SIO · 88-ACR · 88-DCDD · 88-MDS · 88-VI/RTC · 88-C700 · front panel · memory · 8080 CPU · Z80 CPU · VDM-1 · Processor Technology Sol-PC · Host Bridge (our own design).

CP/M 2.2 cold-boots from both 8-inch and 5.25-inch disks. MITS 4K and 8K BASIC and Programming System II load from cassette — including **real `.WAV` audio, played and recorded**, at measured CUTS/ACR parameters.

Debugging, and an assistant that can drive it

Breakpoints, watchpoints, tracepoints, conditional breaks (`BREAK … IF`), execution history, and symbolic reference loaded from `.PRN` and `.SYM` files — DDT and SID run under it, self-modifying RST 7 breakpoints and all. An **MCP server** lets an AI assistant drive a running guest.